

Markov Logic Networks

A Markov Logic Network (MLN) is a way to combine first-order logic (rules about the world) with probability (handling uncertainty)

First-Order Logic:

First-order logic (FOL) is a formal language for representing knowledge about objects, their properties, and the relations between them. It extends propositional logic (which only deals with true/false statements) by adding the ability to talk about *individual objects* and to make general statements using *quantifiers*.

The Four Building Blocks:

Constants name specific objects in the domain. Examples: Anna, Bob, Seattle. Each constant refers to one particular thing.

Variables stand in for objects, without naming any specific one. Examples: x , y , z . Variables let you make general statements like "for every person x ..."

Functions map tuples of objects to other objects. `MotherOf(Anna)` returns Anna's mother. `GreatestCommonDivisor(x, y)` returns a number. A function applied to objects gives you back an object.

Predicates represent relations among objects or attributes of objects. `Friends(Anna, Bob)` is a relation; `Smokes(Anna)` is an attribute. Predicates evaluate to true or false.

Connectives:

You combine formulas using logical connectives. If F_1 and F_2 are formulas:

- $\neg F_1$ (negation): true iff F_1 is false
- $F_1 \wedge F_2$ (conjunction, "and"): true iff both are true
- $F_1 \vee F_2$ (disjunction, "or"): true iff at least one is true
- $F_1 \Rightarrow F_2$ (implication): true iff F_1 is false or F_2 is true
- $F_1 \Leftrightarrow F_2$ (equivalence): true iff both have the same truth value

Quantifiers:

$\forall x F$ (universal quantification, "for all"): true iff F is true for *every* object x in the domain. Example: $\forall x \text{ Smokes}(x) \Rightarrow \text{Cancer}(x)$ says "everyone who smokes gets cancer."

$\exists x F$ (existential quantification, "there exists"): true iff F is true for *at least one* object x . Example: $\exists x \text{ Friends}(\text{Anna}, x)$ says "Anna has at least one friend."

Ground Atoms and Possible Worlds:

Ground Atoms

Think of a predicate like $\text{Friends}(x, y)$ as a template with blanks. The variables x and y are the blanks.

A ground atom is what you get when you fill in *all* the blanks with specific names:

- $\text{Friends}(x, y) \rightarrow$ not ground (blanks still empty)
- $\text{Friends}(\text{Anna}, y) \rightarrow$ not ground (one blank still empty)
- $\text{Friends}(\text{Anna}, \text{Bob}) \rightarrow$ ground! (every blank filled with a real person)

That's it. "Ground" just means *fully filled in with specific objects, no variables left*.

World = a complete snapshot of what's true and false.

A world isn't a machine that returns objects. It's more like a giant **table** that says "here's the status of every fact":

$\text{Smokes}(\text{Anna}) \rightarrow \text{true}$

$\text{Smokes}(\text{Bob}) \rightarrow \text{false}$

$\text{Friends}(\text{Anna}, \text{Bob}) \rightarrow \text{true}$

$\text{Cancer}(\text{Anna}) \rightarrow \text{false}$

Knowledge Bases and Inference:

A knowledge base (KB) is just a collection of facts and rules you believe to be true. That's it

For example, your KB might contain:

1. $\forall x \text{ Smokes}(x) \Rightarrow \text{Cancer}(x)$ ("everyone who smokes gets cancer")
2. $\text{Smokes}(\text{Anna})$ ("Anna smokes")

Limits

First-order logic is enormously expressive, but it has two big practical limitations: inference is only *semi-decidable* in general (an algorithm might never terminate when the answer is "no"), and pure logic is *brittle* formulas that are usually true but occasionally false (like the friends example above) can't be naturally accommodated. A single contradiction makes a KB derive anything at all.

You write down rules about your world in plain logic:

- "Smokers get cancer"
- "Friends tend to have similar smoking habits"
- "Friends of friends are friends"

Problem: these aren't *always* true. Some smokers never get cancer. Some friends have opposite habits. Pure logic breaks when rules have exceptions.

Add Weights

MLNs fix this by attaching a weight to each rule:

Markov Logic Networks:

In traditional first-order logic, formulas are *hard constraints* if a world violates even one formula, it's impossible. MLNs soften this: when a world violates a formula, it just becomes *less probable*, not impossible. The fewer formulas a world violates, the more probable it is.

Each formula gets a **weight** that reflects how strong the constraint is. Higher weight = bigger probability gap between worlds that satisfy the formula and those that don't.

An MLN is formally a set of pairs (F_i, w_i) , where F_i is a first-order formula and w_i is a real-valued weight. Combined with a finite set of constants (objects in your domain), it defines a Markov network in two steps:

That weight measures how strong the rule is:

- **High weight** → big penalty for violating this rule (worlds that break it are much less probable)
- **Low weight** → small penalty (rule is a weak tendency)
- **Infinite weight** → back to hard logic (rule must hold)
- **Zero weight** → rule has no effect

MLN — formulas + weights.

Inference

how do you compute "P(Bob has cancer given Anna smokes)" without enumerating every possible world (which would be impossibly many)?

Key ideas to cover:

- The two-phase approach: build only the part of the network you need, then sample
- **Gibbs sampling** — flip one variable at a time based on its neighbors
- **MaxWalkSat** — find good starting points
- Why this is approximate (and why that's okay)

If you're asking about Bob's cancer, you don't care about facts involving people on the other side of the country. You only care about Bob, his friends, *their* friends, and so on — the chain of influence that reaches Bob.

So the first step is to **build a mini-network** containing only the relevant pieces:

1. Start with the atoms in your question (e.g., Cancer(Bob))
2. Add its **neighbors** — atoms connected to it through some rule
3. Add their neighbors
4. Keep going until you hit only "known" atoms (the evidence)
5. Stop

This is like tracing influence outward from your question. The paper calls this **ConstructNetwork**.

Step 2: Sample From the Mini-Network

Even the mini-network usually has too many possible worlds to enumerate. So we **sample** — generate lots of plausible worlds randomly and use them to estimate probabilities.

The technique is called **Gibbs sampling**, and it works like this:

Gibbs Sampling

Imagine you have 100 binary variables (true/false) and you want to know how often each is true in the "high-scoring" worlds.

1. **Start somewhere.** Just pick any assignment — every atom true, or random, or whatever.
2. **Pick one atom.** Say, `Smokes(Anna)`.
3. **Look at just its neighbors.** Hold everything else fixed.
4. **Flip a weighted coin.** Compute: "Given my neighbors' current values, how likely is it that `Smokes(Anna)` is true vs. false?" Use the MLN's weights to figure out the probabilities. Flip accordingly.
5. **Move on.** Pick another atom. Repeat.

You ask: "P(Cancer(Bob) | Anna smokes, Anna and Bob are friends)?"



Step 1: Build mini-network around Cancer(Bob)

(pull in relevant neighbors, stop at evidence)



Step 2: Use MaxWalkSat to find a good starting world



Step 3: Run Gibbs sampling — flip one atom at a time
based on its neighbors, thousands of times



Step 4: Count how often Cancer(Bob) was true



Report: $\approx 73\%$ probability

Learning

You know weights exist. Next: how do you learn them from a database of facts?

You have a bunch of logical rules. You have a database of facts. **How do you figure out what weight each rule should have?**

You *could* guess weights manually, but that's brittle and doesn't scale. Better: let the data tell you.

You give the MLN:

1. **A set of rules** (formulas in first-order logic)
2. **A database** of known true facts about a domain

For example, the database might say:

- Smokes(Anna) is true
- Smokes(Bob) is true
- Cancer(Anna) is true
- Friends(Anna, Bob) is true

Closed-world assumption: Anything NOT in the database is assumed false. If Cancer(Bob) isn't listed, the system treats it as false.

The problem: computing "how often the model expects this rule to be satisfied" requires running **inference** over the whole network. And as we just learned, inference is expensive — Gibbs sampling, thousands of samples, etc.

Doing full inference at every step of gradient descent? Too slow. The paper tried this and it didn't converge in any reasonable time.

Pseudo-Likelihood

Instead of asking "what's the probability of the entire database?" (which requires global inference), ask a **local question** for each atom:

"Given everything else I know in the database, what's the probability of THIS atom being true?"

Then multiply these local probabilities together. That's pseudo-likelihood.

$$P^*(X) = \prod_l P(X_l \mid \text{neighbors of } X_l)$$

Why this is much faster: Each local probability only needs the atom's **Markov blanket** — its immediate neighbors. You never have to reason about the whole network at once. No global inference required.

The trade-off: Pseudo-likelihood is an *approximation* to true likelihood. It can be slightly biased (especially when atoms have strong long-range dependencies). But in practice, it works surprisingly well, and it makes learning practical instead of impossible.

What the Algorithm Actually Does

The full learning loop:

1. **Count once.** For each rule, count how often it's satisfied in the data, and how often it would be satisfied if each individual atom were flipped. These counts don't change during training, so compute them once upfront.
2. **Optimize with L-BFGS.** L-BFGS is a standard numerical optimizer (think: smart gradient descent). It iteratively adjusts the weights to maximize pseudo-likelihood.
3. **Add a prior.** To prevent overfitting, the algorithm penalizes very large weights (using a Gaussian prior centered at zero). This keeps weights reasonable even when data is sparse.

Learning the Rules Themselves

So far we've assumed you provide the rules and only the weights are learned. But what if you don't know the rules?

The paper uses an Inductive Logic Programming (ILP) system called **CLAUDIEN** to discover rules automatically from the data. You give it:

- The database
- A "language bias" (constraints like "rules can have at most 3 variables")

The Experiments

How MLNs compare to pure logic and pure probability approaches.

Applications

Five concrete tasks expressed as MLNs:

- Collective classification
- Link prediction
- Link-based clustering
- Social network modeling
- Object identification (entity resolution)

Work Flow:

1. What is the Network?

In MLN, the **network** is made of nodes and edges.

A **node** is one fact that can be true or false.

Example:

$$\textit{Smokes}(\textit{Anna})$$

This is one node.

It can be:

$$\textit{Smokes}(\textit{Anna}) = 1$$

meaning Anna smokes,

or:

$$\textit{Smokes}(\textit{Anna}) = 0$$

meaning Anna does not smoke.

Other nodes can be:

$$\begin{aligned} &\textit{Cancer}(\textit{Anna}) \\ &\textit{Friends}(\textit{Anna}, \textit{Bob}) \\ &\textit{Smokes}(\textit{Bob}) \end{aligned}$$

So the network is basically a graph of facts.

2. How Edges Are Created

Two nodes are connected if they appear in the same rule.

Example rule:

$$\textit{Smokes}(x) \Rightarrow \textit{Cancer}(x)$$

After grounding for Anna:

$$\textit{Smokes}(\textit{Anna}) \Rightarrow \textit{Cancer}(\textit{Anna})$$

Now these two nodes are connected:

$$\begin{aligned} &\textit{Smokes}(\textit{Anna}) \\ &\textit{Cancer}(\textit{Anna}) \end{aligned}$$

Why?

Because the truth of one affects the probability of the other.

If we know Anna smokes, then the probability of cancer may increase.

3. Simple Network Example

Suppose we have this rule:

$$\textit{Smokes}(x) \Rightarrow \textit{Cancer}(x)$$

And constants:

$$C = \{\textit{Anna}, \textit{Bob}\}$$

Grounding creates:

$$\begin{aligned}\textit{Smokes}(\textit{Anna}) &\Rightarrow \textit{Cancer}(\textit{Anna}) \\ \textit{Smokes}(\textit{Bob}) &\Rightarrow \textit{Cancer}(\textit{Bob})\end{aligned}$$

So the network has nodes:

$$\begin{aligned}\textit{Smokes}(\textit{Anna}) \\ \textit{Cancer}(\textit{Anna}) \\ \textit{Smokes}(\textit{Bob}) \\ \textit{Cancer}(\textit{Bob})\end{aligned}$$

And edges:

$$\begin{aligned}\textit{Smokes}(\textit{Anna}) - \textit{Cancer}(\textit{Anna}) \\ \textit{Smokes}(\textit{Bob}) - \textit{Cancer}(\textit{Bob})\end{aligned}$$

Simple meaning:

The general rule creates specific connections in the network.

4. What Does Weight Mean?

Each rule has a weight.

Example:

$$Smokes(x) \Rightarrow Cancer(x)$$

Weight:

$$w = 1.5$$

The weight tells how strong the rule is.

If weight is high, the rule strongly affects probability.

If weight is low, the rule weakly affects probability.

If weight is infinite, the rule becomes a hard rule.

So:

High weight $\Rightarrow$ Strong rule
Low weight $\Rightarrow$ Weak rule
Infinite weight $\Rightarrow$ Hard logic

5. Main Formula

The main MLN formula is:

$$P(X = x) = \frac{1}{Z} \exp \left(\sum_i w_i n_i(x) \right)$$

This formula calculates the probability of one possible world.

6. Meaning of Each Term

$$P(X = x)$$

This means:

Probability that the complete world is x .

A world means one full assignment of true or false values to all facts.

Example world:

Smokes(Anna) = True
Cancer(Anna) = True

$Smokes(Bob) = False$
 $Cancer(Bob) = False$

The MLN gives this full world a probability.

Z

Z

is the **partition function**.

Simple meaning:

Z is used to normalize the probabilities.

It makes sure that probabilities of all possible worlds add up to 1.

Without Z , we only have scores, not valid probabilities.

exp

$\exp(\dots)$

means exponential.

Simple meaning:

It turns the score into a positive number.

Probability scores must be positive, so MLN uses exponential.

$\sum_i$

$\sum_i$

means add over all formulas.

If the MLN has 4 rules, then it adds the contribution of all 4 rules.

Example:

$$w_1 n_1(x) + w_2 n_2(x) + w_3 n_3(x) + w_4 n_4(x)$$

$$w_i$$

$$w_i$$

means weight of rule i .

Example:

$$w_1 = 1.5$$

for:

$$Smokes(x) \Rightarrow Cancer(x)$$

This tells how important rule 1 is.

$$n_i(x)$$

$$n_i(x)$$

means number of true groundings of rule i in world x .

Example rule:

$$Smokes(x) \Rightarrow Cancer(x)$$

Constants:

$$C = \{Anna, Bob\}$$

Grounded rules:

$$Smokes(Anna) \Rightarrow Cancer(Anna)$$

$$Smokes(Bob) \Rightarrow Cancer(Bob)$$

If both grounded rules are true in world x , then:

$$n_i(x) = 2$$

If only one is true:

$$n_i(x) = 1$$

If none are true:

$$n_i(x) = 0$$

Simple meaning:

$n_i(x)$ counts how many times the rule is satisfied in that world.

7. How the Formula Works

The inside part is:

$$\sum_i w_i n_i(x)$$

This is the world score.

If a world satisfies many high-weight rules, this score becomes large.

Then:

$$\exp\left(\sum_i w_i n_i(x)\right)$$

turns that score into a positive value.

Then:

$$\frac{1}{Z}$$

normalizes it into a real probability.

Simple meaning:

Satisfied strong rules $\Rightarrow$ Big score $\Rightarrow$ High probability
Violated strong rules $\Rightarrow$ Small score $\Rightarrow$ Low probability

8. Full Workflow of MLN

Step 1: Write Logical Rules

First, we write rules.

Example:

$Smokes(x) \Rightarrow Cancer(x)$
 $Friends(x, y) \Rightarrow (Smokes(x) \Leftrightarrow Smokes(y))$

The first rule says smokers are more likely to have cancer.

The second rule says friends tend to have similar smoking habits.

Step 2: Assign Weights

Each rule gets a weight.

Example:

$$\begin{aligned} \textit{Smokes}(x) &\Rightarrow \textit{Cancer}(x), w = 1.5 \\ \textit{Friends}(x, y) &\Rightarrow (\textit{Smokes}(x) \Leftrightarrow \textit{Smokes}(y)), w = 1.1 \end{aligned}$$

The first rule is stronger because 1.5 is greater than 1.1.

Step 3: Give Constants

Now we give actual objects.

Example:

$$C = \{\textit{Anna}, \textit{Bob}\}$$

These are the people in our domain.

Step 4: Ground the Rules

Replace variables with constants.

General rule:

$$\textit{Smokes}(x) \Rightarrow \textit{Cancer}(x)$$

Grounded rules:

$$\begin{aligned} \textit{Smokes}(\textit{Anna}) &\Rightarrow \textit{Cancer}(\textit{Anna}) \\ \textit{Smokes}(\textit{Bob}) &\Rightarrow \textit{Cancer}(\textit{Bob}) \end{aligned}$$

General rule:

$$\textit{Friends}(x, y) \Rightarrow (\textit{Smokes}(x) \Leftrightarrow \textit{Smokes}(y))$$

Grounded version:

$$\textit{Friends}(\textit{Anna}, \textit{Bob}) \Rightarrow (\textit{Smokes}(\textit{Anna}) \Leftrightarrow \textit{Smokes}(\textit{Bob}))$$

Step 5: Create Nodes

Every ground atom becomes a node.

Examples:

Smokes(Anna)
Cancer(Anna)
Smokes(Bob)
Cancer(Bob)
Friends(Anna, Bob)

Each node is true or false.

Step 6: Create Edges

If ground atoms appear in the same grounded rule, connect them.

Example:

Smokes(Anna) $\Rightarrow$ Cancer(Anna)

creates an edge between:

Smokes(Anna)

and:

Cancer(Anna)

Another example:

Friends(Anna, Bob) $\Rightarrow$ (Smokes(Anna) $\Leftrightarrow$ Smokes(Bob))

connects:

Friends(Anna, Bob)
Smokes(Anna)
Smokes(Bob)

Step 7: Generate Possible Worlds

The network considers possible truth assignments.

Example world 1:

Smokes(Anna) = True

$Cancer(Anna) = True$
 $Smokes(Bob) = False$
 $Cancer(Bob) = False$

Example world 2:

$Smokes(Anna) = True$
 $Cancer(Anna) = False$
 $Smokes(Bob) = False$
 $Cancer(Bob) = False$

World 1 satisfies the smoking-cancer rule for Anna.

World 2 violates it.

So world 1 gets higher probability than world 2.

Step 8: Count True Groundings

For each world, count how many grounded rules are true.

Example:

$Smokes(Anna) \Rightarrow Cancer(Anna)$

If:

$Smokes(Anna) = True$
 $Cancer(Anna) = True$

then the rule is true.

If:

$Smokes(Anna) = True$
 $Cancer(Anna) = False$

then the rule is false.

So for each rule, MLN calculates:

$n_i(x)$

Step 9: Calculate World Score

The score is:

$$\sum_i w_i n_i(x)$$

Example:

Suppose:

$$\begin{aligned} w_1 &= 1.5 \\ n_1(x) &= 2 \end{aligned}$$

Then contribution is:

$$w_1 n_1(x) = 1.5 \times 2 = 3$$

If another rule has:

$$\begin{aligned} w_2 &= 1.1 \\ n_2(x) &= 1 \end{aligned}$$

Then contribution is:

$$w_2 n_2(x) = 1.1 \times 1 = 1.1$$

Total score:

$$3 + 1.1 = 4.1$$

Step 10: Convert Score to Probability

Now MLN uses:

$$P(X = x) = \frac{1}{Z} \exp(\text{score})$$

Since:

$$\text{score} = \sum_i w_i n_i(x)$$

we get:

$$P(X = x) = \frac{1}{Z} \exp \left(\sum_i w_i n_i(x) \right)$$

Simple meaning:

Higher score means higher probability.

9. Inference Workflow

Inference means asking a question.

Example question:

What is the probability that Anna has cancer if Anna smokes?

$$P(\text{Cancer}(\text{Anna}) \mid \text{Smokes}(\text{Anna}))$$

The workflow is:

First, set the evidence:

$$\text{Smokes}(\text{Anna}) = \text{True}$$

Then ask for the query:

$$\text{Cancer}(\text{Anna}) = ?$$

The MLN checks possible worlds where Anna smokes.

Among those worlds, it checks how many likely worlds also have Anna with cancer.

That gives:

$$P(\text{Cancer}(\text{Anna}) \mid \text{Smokes}(\text{Anna}))$$

Simple meaning:

Inference means using known facts to estimate unknown facts.

10. Inference Formula

The inference formula is:

$$P(F_1 \mid F_2, L, C) = \frac{\sum_{x \in X_{F_1} \cap X_{F_2}} P(X = x \mid M_{L,C})}{\sum_{x \in X_{F_2}} P(X = x \mid M_{L,C})}$$

Where:

F_1 = query

F_2 = evidence

L = MLN

C = constants

$M_{L,C}$ = ground Markov network

X_{F_1} = worlds where query is true

X_{F_2} = worlds where evidence is true

Simple meaning:

The denominator looks at all worlds where evidence is true.

The numerator looks at worlds where both evidence and query are true.

Then the ratio gives the probability of the query given evidence.

11. Learning Workflow

Learning means finding good weights.

The formulas may already be known.

Example formula:

$$Smokes(x) \Rightarrow Cancer(x)$$

But we may not know its weight.

So the MLN learns the weight from data.

The workflow is:

First, observe data.

Example:

Smokes(Anna) = True

Cancer(Anna) = True

Smokes(Bob) = True

Cancer(Bob) = False

Then count how often each rule is true.

If a rule is true often, increase its weight.

If a rule is false often, decrease its weight.

12. Learning Formula

The log-likelihood gradient is:

$$\frac{\partial}{\partial w_i} \log P_w(X = x) = n_i(x) - \sum_{x'} P_w(X = x') n_i(x')$$

Where:

w_i = weight of formula i

$n_i(x)$ = number of true groundings in real data

$\sum_{x'} P_w(X = x') n_i(x')$ = expected number of true groundings according to the model

Simple meaning:

$$\text{Gradient} = \text{Real count} - \text{Model expected count}$$

If real count is bigger than model expectation, increase the weight.

If real count is smaller than model expectation, decrease the weight.

13. Example

Suppose rule is:

$$\text{Smokes}(x) \Rightarrow \text{Cancer}(x)$$

Weight:

$$w = 1.5$$

Constants:

$$C = \{Anna\}$$

Grounded rule:

$$\text{Smokes}(Anna) \Rightarrow \text{Cancer}(Anna)$$

Now possible worlds:

World 1:

$$\text{Smokes}(Anna) = \text{False}, \text{Cancer}(Anna) = \text{False}$$

World 2:

$$\text{Smokes}(Anna) = \text{False}, \text{Cancer}(Anna) = \text{True}$$

World 3:

Smokes(Anna) = True, Cancer(Anna) = False

World 4:

Smokes(Anna) = True, Cancer(Anna) = True